



CLASS RELATIONS

Object-Oriented Programming

Politeknik Negeri Malang

Overview, Agenda & Learning Outcomes

After this theory session, students will be able to understand Uses-A / Has-A relations (Composition & Aggregation), delegation between objects, arrays of objects, and apply defensive programming to relations between classes.

PART 1 — Concepts & Foundations (50 minutes)

Object Relations, Three Types of Relations, Composition vs Aggregation, Memory Model

PART 2 — Case Studies / Live Coding (110 minutes)

Laptop→Processor (+Printer/Uses-A), Character→Weapon, Student→Department

PART 3 — Error Handling & Design (40 minutes)

Pitfalls, Defensive Programming, Relation Design Heuristics

CLOSING — Summary, Practicum Preview, Q&A (20 minutes)

Object Relations: Basic Concepts

- So far, the attributes of the classes you have built have been of simple (primitive) type, such as double, int, float, etc.
- A class can have an attribute typed as another class — in other words, that attribute is an object of another class (an object-typed attribute)
- A Library can have many Book as its attribute, and a Car can have an Engine as its attribute
- In more complex cases, a system will contain more than one class that is interrelated with other classes.
- While most of the cases worked on in previous experiments focused on only a single class, this material will discuss example cases involving several classes that are related to one another.

Object Relations: Basic Concepts

- Objects in the real world **rarely stand alone** → — an object is usually composed of, or depends on, other objects.
- In OOP, this is modeled by letting an attribute of a class be typed as another class → — it does not have to be a primitive type like `int` or `String`.
- Example:
 - class `Airplane`, which contains the objects `Engine`, `Wing`, `Fin`, and `Pilot`.
 - class `Laptop`, which contains a `Processor`.
- For an object to **"have"** another object, the owned object must already be created first → before it is linked/attached (we will see how, via a *constructor* or *setter*).

→ This concept is what we call a HAS-A relation: a class "has" an object of another class as its attribute.

Why Are Object Relations Important?

✗ Without good relation design:

- All attributes are piled into a single class → hard to maintain
- Data structure is unclear → hard for a team to develop

✓ With good relation design:

- Each class has a specific responsibility (single responsibility)
- Code is more reusable, easier to test, and easier to develop

Relations between objects are the foundation of good object-oriented system design → used in nearly every real-world application.

Three Types of Relations Between Classes

1. **USES-A** (*Dependency*) → temporary/weak relation
 - The other object is only used briefly, **NOT stored as an attribute**
 - Example: method `Driver.drive(Car c)`
2. **HAS-A** (*Association / Composition / Aggregation*)
 - The other object **IS STORED as an attribute** (*field*) permanently
 - Example: Laptop HAS-A Processor
3. **IS-A** (*Inheritance*) → **will be discussed in depth in another session**

Example: Dog IS-A Animal

? A quick way to tell them apart in your own code:

Is the object stored as a private field in the class? → HAS-A. Does the object only appear as a method parameter and is not stored? → USES-A.

A contrasting example with the SAME class: `Driver.drive(Car c)` → USES-A (parameter). But if `Driver` has `private Car currentCar`; → that is already HAS-A (Association) — even though it is **NOT a whole-part** relation like *Composition/Aggregation*.

Spectrum of HAS-A Relation Strength

ASSOCIATION (Weakest)

Merely "knows & stores" a reference
Not whole-part, without special ownership
The part stands on its own, not bound

Example:

Driver — Car

(Driver stores a reference to Car, but Car is not "part of" Driver)

Code:

```
private Car currentCar;
```

AGGREGATION ◇ (Weak Ownership)

Whole-part, but the part is independent
If the "whole" is deleted, the "part" still exists
The part can be shared/moved to another whole

Example:

Student ◇ Department

Laptop ◇ Processor (injected via setCPU — see Case Study 1)

Code:

```
Student(..., Department dept){  
    Dept = dept;  
}
```

COMPOSITION ◆ (Strong Ownership)

Whole-part, the part's lifecycle is bound to the whole
If the "whole" is deleted, the "part" is destroyed with it
The part is usually not shared

Example:

Car ◆ Engine

(Engine is created directly inside the Car constructor, "dies" together with Car)

Code:

```
Engine engine = new Engine();
```

All three are equally HAS-A (a stored field) → — the difference is only HOW STRONG the ownership is.

Characteristics in Java: Code Signature of Each Relation

Relation Type	Logical Relationship	Code Signature in Java	Strength
Dependency	Uses-a (temporary use)	Appears as a parameter or local variable inside a method	1/4 (Weakest)
Association	Peer-to-peer (mutually aware)	Field stores a REFERENCE to an object belonging to another class (stored, not created)	2/4
Aggregation ◇	Has-a (loosely owns)	The "part" object is created OUTSIDE, then injected via a constructor/setter	3/4
Composition ◆	Part-of (an absolute part)	The "part" object is created DIRECTLY inside the parent class (usually in the constructor)	4/4 (Strongest)

A Real Composition Example: Car ♦ Engine

```
public class Engine
{
    private String type;

    public Engine()
    {
        this.type = "4-cylinder";
    }
    public String getType()
    {
        return type;
    }
}

public class Car
{
    private String brand;
    private Engine engine;

    public Car(String brand)
    {
        this.brand = brand;
        this.engine = new Engine();
    }
    public void displayInfo()
    {
        System.out.println("Car: " + brand);
        System.out.println("Engine: " + engine.getType());
    }
}
```

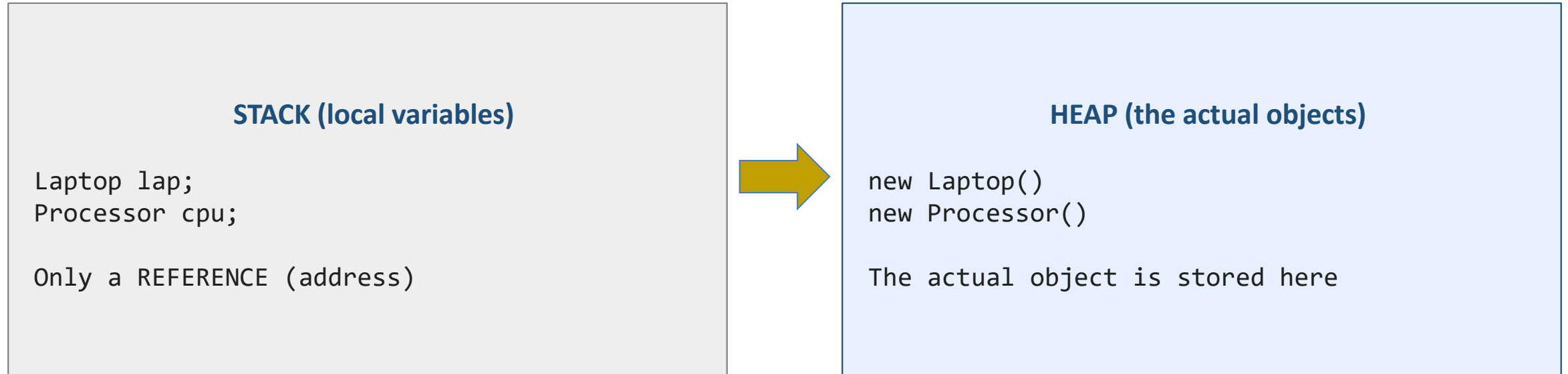
Engine is created DIRECTLY inside the constructor of Car
→, not received from outside as a parameter.

There is no `setEngine()/setEngine()`. Once Car is created, its Engine cannot be replaced or retrieved from outside.

✓ This is what enforces Composition at the CODE level:
the lifecycle of Engine is fully controlled by Car.

Compare this with Laptop ◇ Processor (Aggregation):
Processor is created OUTSIDE, then injected via `setCPU()`.

Memory Model: Stack vs Heap

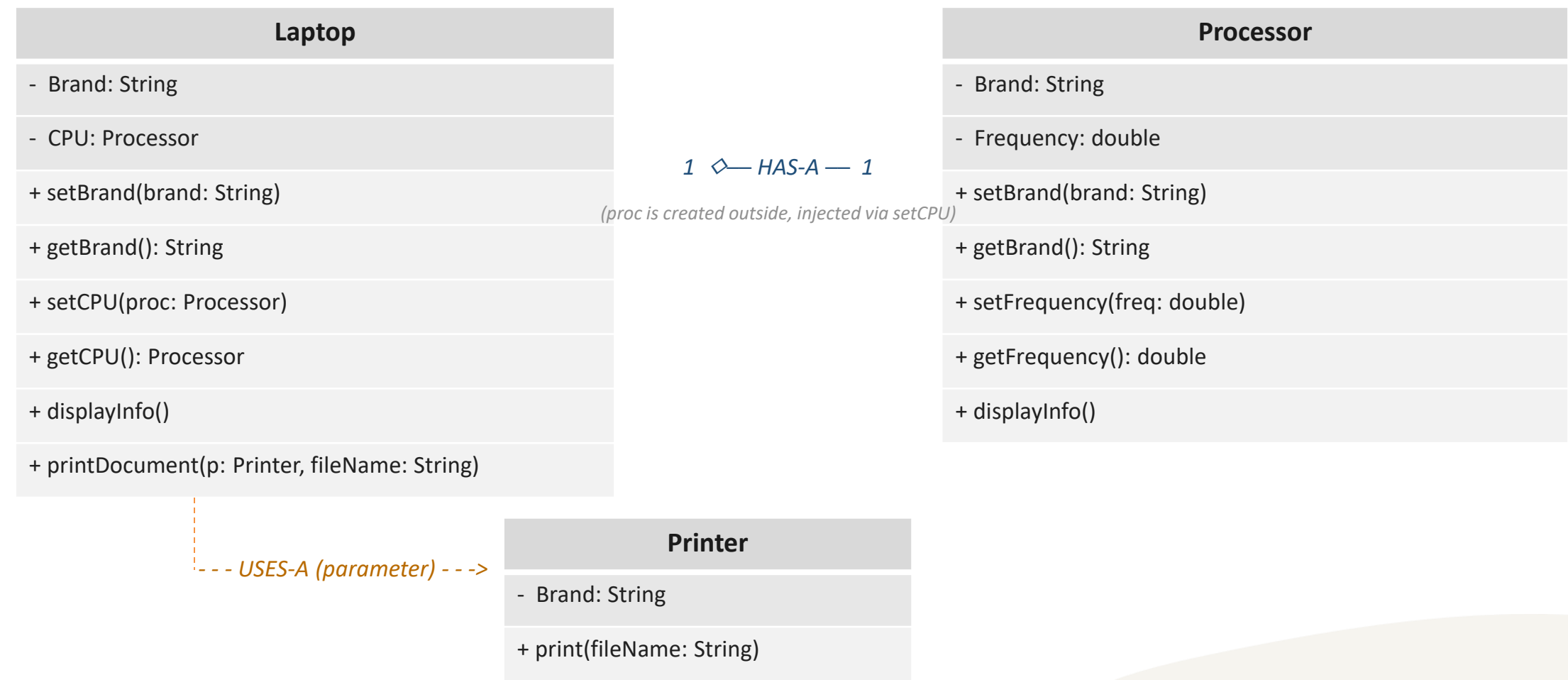


IMPORTANT: `lap.setCPU(cpu)` copies the REFERENCE, it does not duplicate the Processor object.

The same applies when an object is passed as a method parameter (e.g., `printDocument(Printer p)`): what gets copied is the REFERENCE. Changing the contents of the object inside the method will be visible outside; but reassigning the parameter variable to a new object does NOT affect the caller's original variable.

A collection of objects interconnected via references like this is called an object graph — a term that will come up often when discussing relations between objects (e.g., `Laptop → Processor`, or `Student → Department` shared by many).

Case Study 1: Laptop ◇ Processor (+ Uses-A Printer)



Diamond ◇ = HAS-A/Aggregation (field, injected from outside). Dashed line = USES-A (parameter only, not stored) → — we will prove this in the code shortly.

Implementation: class Processor

```
public class Processor
{
    private String Brand;
    private double Frequency;

    public String getBrand()
    {
        return Brand;
    }
    public void setBrand(String brand)
    {
        Brand = brand;
    }
    public double getFrequency()
    {
        return Frequency;
    }
    public void setFrequency(double freq)
    {
        Frequency = freq;
    }
    public void displayInfo()
    {
        System.out.println("Processor brand: " + Brand);
        System.out.println("Processor frequency: " +
            Frequency + " GHz");
    }
}
```

- Class Processor stands on its own.
- It has private attributes Brand and Frequency.
- Getters/setters and a displayInfo() method are provided to display its own data.

Implementation: class Laptop (HAS-A Processor)

```
public class Laptop
{
    private String Brand;
    private Processor CPU;

    public String getBrand()
    {
        return Brand;
    }
    public void setBrand(String brand)
    {
        Brand = brand;
    }
    public Processor getCPU()
    {
        return CPU;
    }
    public void setCPU(Processor proc)
    {
        CPU = proc;
    }
    public void displayInfo()
    {
        System.out.println("Laptop brand: " + Brand);
        CPU.displayInfo();
    }
}
```

- Notice the attribute:
private Processor CPU;
- This is the HAS-A relation: Laptop "has" a Processor as part of itself.
- setCPU(proc) receives a REFERENCE to a Processor object that has already been created beforehand.

✓ Why Aggregation, Not Composition

Processor is created OUTSIDE the Laptop class, then injected via setCPU() — it is not instantiated directly inside Laptop.

Running: TestLaptop

```
public class TestLaptop
{
    public static void main(String[] args)
    {
        Processor proc = new Processor();
        proc.setBrand("Intel Core i7");
        proc.setFrequency(2400);

        Laptop lap = new Laptop();
        lap.setBrand("Asus");
        lap.setCPU(proc);

        lap.displayInfo();
    }
}
```

Program output

```
G:\Java>java TestLaptop
```

```
Laptop brand: Asus
Processor brand: Intel Core i7
Processor frequency: 2400.0 GHz
```

Steps:

1. Create a Processor object, fill in its data
2. Create a Laptop
3. Connect: `lap.setCPU(proc)`
4. `lap.displayInfo()` automatically calls `CPU.displayInfo()` → delegation between objects

Accessing Nested Attributes (Chaining)

```
public class TestLaptop
{
    public static void main(String[] args)
    {
        Processor proc = new Processor();
        proc.setBrand("Intel Core i7");
        proc.setFrequency(2400);
        Laptop lap = new Laptop();
        lap.setBrand("Asus");
        lap.setCPU(proc);
        System.out.println("Laptop Brand " + lap.getBrand());
        System.out.println("with processor " + lap.getCPU().getBrand());
    }
}
```

⚠ Two risks of `lap.getCPU().getBrand()`:

1. If `getCPU()` returns null, this line CRASHES (NullPointerException).
2. Code OUTSIDE can reach into CPU and change it directly, for example: `lap.getCPU().setFrequency(9999)`; ← *Laptop never knows this happened!*

This violates *Law of Demeter* ("don't talk to strangers") and risks *encapsulation leak*. The solution is on the next slide.

Addressing Risk #1: NullPointerException

✗ PROBLEM (will crash):

```
Laptop lap = new Laptop();  
lap.setBrand("Dell");  
// CPU was never set (still null)  
lap.displayInfo(); // CRASH! CPU.displayInfo()
```

✓ SOLUTION (null check inside displayInfo):

```
public void displayInfo() {  
    System.out.println("Laptop brand: " + Brand);  
    if (CPU != null) {  
        CPU.displayInfo();  
    } else {  
        System.out.println("Processor: not installed");  
    }  
}
```

Addressing Risk #2: Tell, Don't Ask

Laptop
- Brand: String
- CPU: Processor
+ setBrand/getBrand
+ setCPU/getCPU
+ displayInfo()
+ overclock(): void ← NEW

Processor
- Brand: String
- Frequency: double
+ setFrequency/getFrequency
+ displayInfo()

Principle: TELL, DON'T ASK

Instead of outside code "asking" (`getCPU()`) and then changing it directly, it is better for Laptop to be given a "command" (`overclock()`) and for Laptop to handle its own internal details:

```
public void overclock() { CPU.setFrequency(CPU.getFrequency() + 100); }
```

Important note: this does not mean chaining is ALWAYS bad. Laptop MAY call `CPU.setFrequency()` inside its own method → — that still stays within the bounds of its own class.

What must be avoided is CODE OUTSIDE class Laptop reaching directly into CPU.

Uses-A in Code: Laptop Uses a Printer

Printer is NOT an attribute of Laptop — it only passes through as a method parameter → USES-A.

```
public class Printer {  
    private String Brand;  
    public Printer(String brand) { Brand = brand; }  
    public void print(String fileName) {  
        System.out.println("Printing " + fileName + " on " + Brand);  
    }  
}  
  
// New method in class Laptop – there is NO Printer field!  
public void printDocument(Printer p, String fileName) {  
    p.print(fileName);  
}  
  
// addition in class TestLaptop – create a printer object and pass it when calling printDocument  
Printer prn = new Printer("Epson L120");  
lap.printDocument(prn, "laporan.pdf");  
// Output: Printing Laporan.pdf on Epson L120
```

Design note: if Laptop prints to the SAME printer FREQUENTLY, consider storing it as a field (Has-A). Uses-A is suited for incidental use only.

Case Study 2: Character ◇ Weapon[] (Array of Objects)

Character
- Name: String
- EquippedWeapons: Weapon[]
+ getName(): String
+ setName(String name)
+ getEquippedWeapons(): Weapon[]
+ setEquippedWeapons(Weapon[] weapons)
+ displayInfo()

1 ◇— HAS-A — 1..* (one or more)

Weapon
- Name: String
- Power: int
+ Weapon(name: String, power: int)
+ getName(): String
+ setName(String name)
+ getPower(): int
+ setPower(power: int)
+ displayInfo()

An object attribute can occur more than once → use an array to store them. Character has many Weapons (aggregation → — created outside, injected via a setter).

Implementation: class Weapon

```
public class Weapon
{
    private String Name;
    private int Power;

    public Weapon(String name, int pwr)
    {
        Name = name;
        Power = pwr;
    }
    public String getName()
    {
        return Name;
    }
    public void setName(String name)
    {
        Name = name;
    }
    public int getPower()
    {
        return Power;
    }
    public void setPower(int k)
    {
        Power = k;
    }
}
```

Class Weapon is similar to Processor: it stands on its own, storing Name and Power.

Constructor `Weapon(name, power)` makes it easier to create an object directly with its initial data already filled in

→ — no need for separate `setName()`/`setPower()` calls.

Implementation: class Character (HAS-A Weapon[])

```
public class Character
{
    private String Name;
    private Weapon EquippedWeapons[];

    public String getName()
    {
        return Name;
    }
    public void setName(String name)
    {
        Name = name;
    }
    public Weapon[] getEquippedWeapons()
    {
        return EquippedWeapons;
    }
    public void setEquippedWeapons(Weapon[] wpns)
    {
        EquippedWeapons = wpns;
    }
    public void displayInfo()
    {
        System.out.println("\nCharacter name: " + Name);
        for (int i = 0; i < EquippedWeapons.length; i++)
        {
            System.out.println("Weapon " + i + ": " +
                               EquippedWeapons[i].getName());
        }
    }
}
```

`private Weapon EquippedWeapons[];` → an array of objects: each element is a REFERENCE to a Weapon object, and they can all be different.

💡 Without overriding `toString()`, printing an object directly (e.g., `print(weapon)`) will display `"Weapon@1b6d358"` → rather than useful information. That is why its own `displayInfo()` method is used instead.

Running: TestCharacter

```
public class TestCharacter
{
    public static void main(String[] args)
    {
        Weapon wpn1 = new Weapon("Sword", 100);
        Weapon wpn2 = new Weapon("Bow", 70);
        Weapon wpn3 = new Weapon("Knife", 30);
        Character char1 = new Character();
        char1.setName("Arjuna");
        char1.setEquippedWeapons(
            new Weapon[]{wpn1, wpn2} );
        Character char2 = new Character();
        char2.setName("Kurawa");
        char2.setEquippedWeapons( new Weapon[]{wpn3} );
        char1.displayInfo();
        char2.displayInfo();
    }
}
```

Program output

G:\Java>java TestCharacter

Character name: Arjuna
Weapon 0: Sword
Weapon 1: Bow

Character name: Kurawa
Weapon 0: Knife

Each Character object has its own
Weapon array — →Arjuna and Kurawa
do not share the same weapons.

`new Weapon[]{wpn1, wpn2}` creates an array
literal containing REFERENCES to objects that
were already created earlier.

Pitfall: Array Index & Unfilled Elements

✗ `ArrayIndexOutOfBoundsException`:

```
Weapon[] wpn = new Weapon[3]; // capacity 3
wpn[5] = new Weapon("Bow", 70); // CRASH!
```

✗ Array element not yet filled (null):

```
Weapon[] wpn = new Weapon[3];
wpn[0] = new Weapon("Sword", 100);
for (int i = 0; i < wpn.length; i++) {
    wpn[i].displayInfo(); // CRASH at i=1! (still null)
}
```

✓ SOLUTION: validate the index & null-check before accessing

```
if (i < wpn.length && wpn[i] != null) {
    wpn[i].displayInfo();
}
```

Defensive Programming Patterns (Summary)

PATTERN 1 → Null Checking

```
if (obj != null) { obj.method(); }
```

PATTERN 2 → Array Bounds Validation

```
if (index >= 0 && index < array.length) { ... }
```

PATTERN 3 → Safe Iteration with a Counter

```
for (int i = 0; i < filledCount; i++) { ... }
```

PATTERN 4 → Defensive Getter

```
if (CPU == null) throw new IllegalStateException("CPU has not been set");
```



Principle: never assume data is always valid → always validate!

Case Study 3: Student ◇ Department (Aggregation)



Unlike Laptop–Processor: Department here is created SEPARATELY and then "lent" to Student via the constructor. Department still exists even if one Student is deleted, and can be shared by many other Students (multiplicity * on the Student side, 1 on the Department side).

Implementation: class Department & Student

```
public class Department {  
    private String Name;  
    private String Accreditation;  
  
    public void setName(String n) {  
        Name = n;  
    }  
    public String getName() {  
        return Name;  
    }  
    public void setAccreditation(String a) {  
        Accreditation = a;  
    }  
    public String getAccreditation() {  
        return Accreditation;  
    }  
}
```

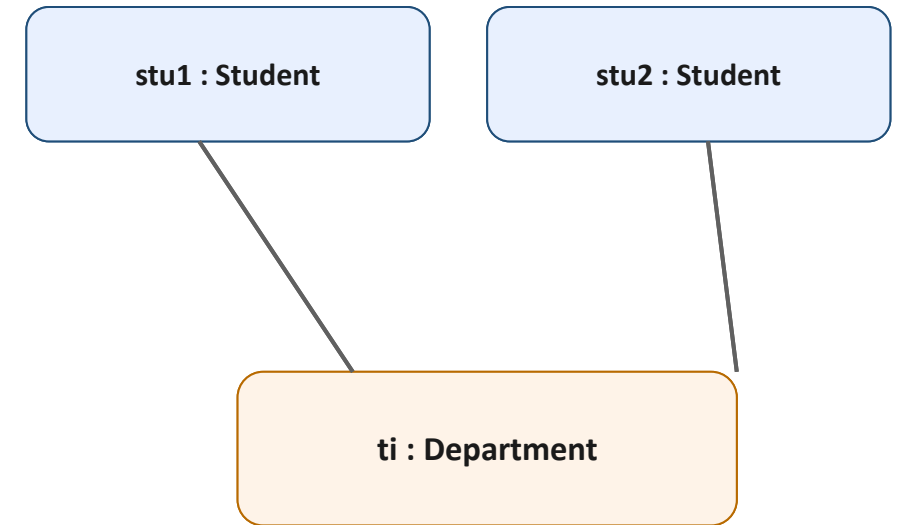
```
public class Student {  
    private String StudentId, Name;  
    private Department Dept; // Has-A (Aggregation)  
  
    public Student(String studentId, String name,  
                   Department dept) {  
        StudentId = studentId; Name = name; Dept = dept;  
    }  
  
    public void displayInfo() {  
        System.out.println("Student ID: " + StudentId);  
        System.out.println("Name: " + Name);  
        System.out.println("Department: " +  
        Dept.getName());  
    }  
}
```

Proving Aggregation: Sharing One Object

```
        Department ti = new Department();
ti.setName("Information Technology");
ti.setAccreditation("A");

Student stu1 = new Student("111001", "Jones", ti);
Student stu2 = new Student("111002", "Alice", ti);
// note: the SAME ti is used by both

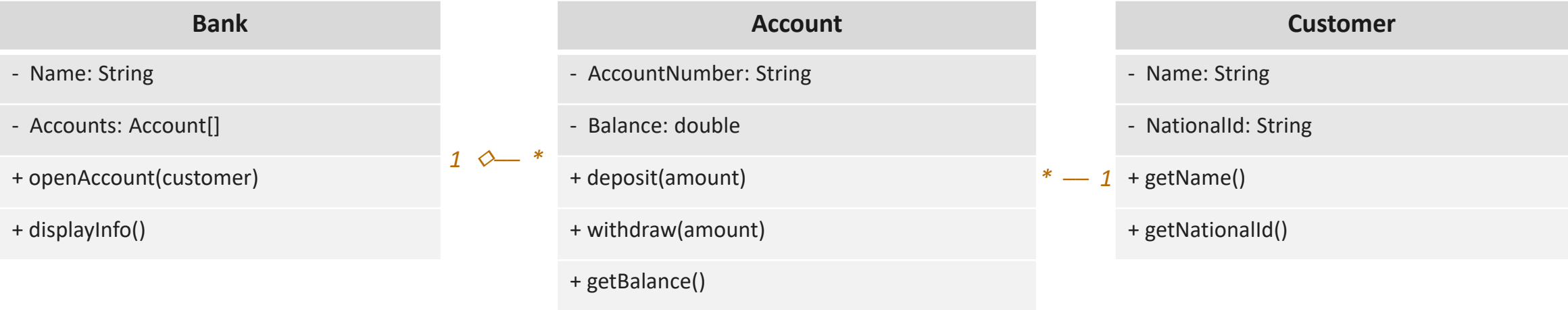
stu1.displayInfo();
stu2.displayInfo();
// Output:
// Student ID: 111001 / Name: Jones / Department:
// Information Technology
// Student ID: 111002 / Name: Alice / Department:
// Information Technology
```



The SAME Department object used by 2 Students

- stu1 & stu2 point to the SAME ti OBJECT, not two separate copies.
- If stu1 is deleted, the ti object STILL EXISTS (still used by stu2).
- Unlike Composition: Engine in Car is created and "dies" together with its own Car — it is not designed to be shared with another Car.

Discussion: Bank ◇ Account ◇ Customer



? Discussion questions (answer using the Student ◇ Department pattern from before):

1. Bank ◇ Account: Aggregation or Composition? If the Bank where an Account was opened CLOSES, does the owner's Account disappear too? (hint: it is usually migrated, not lost)
2. One Customer can have MORE THAN ONE Account (savings + checking). Does this change the type of the Account—Customer relation, or does it only change its multiplicity?
3. Compare this with Laptop ◆ Processor: why is Account—Customer NOT Composition even though both are equally "owned"?

→ Short answer: both are Aggregation — an Account continues to exist independent of any particular Bank/Customer; only the multiplicity differs (1..* on the Account side for Customer).

When Do You Need a Relation? (Design Heuristics)

Questions you can use when designing your own classes:

1. Does this data have its own identity/behavior that stands on its own?

Yes → make it a separate class (a relation). Just a simple value (e.g., int x, y on Point) → a plain attribute is enough, no new class is needed.

2. How many instances are needed, and is that number fixed?

A FIXED and small number (e.g., exactly 2 cameras) → separate attributes: FrontCamera, RearCamera

A DYNAMIC/large number (e.g., a character's weapons) → array/collection

3. Is this object exclusively owned by one owner, or can it be shared?

Exclusive, lifecycle bound → Composition (◆)

Shareable, independent lifecycle → Aggregation (◇)

*Domains we have already seen: Laptop/Processor, Character/Weapon, Student/Department, Driver/Car, Bank/Account/Customer → the same pattern, different context.
Practice recognizing the pattern, not memorizing the examples.*

Summary & Key Takeaways

- ✓ HAS-A (a stored field) → Composition (◆, bound) or Aggregation (◇, shareable)
- ✓ USES-A: the object only passes through as a parameter, not stored as a field
- ✓ Quick heuristic: stored as a field → HAS-A; parameter only → USES-A
- ✓ An object variable stores a REFERENCE; the actual object lives on the Heap
- ✓ Tell, Don't Ask: hide internal details, avoid chaining from outside (Law of Demeter)
- ✓ An array of objects stores many references — make sure all are filled & validated
- ✓ Always apply defensive programming: null checks, bounds/index validation
- ✓ UML diagrams + multiplicity help you design relations before writing code

Individual Exercise

A motorcycle consists of the following attributes: Brand, Color, Engine, and MaxSpeed, as well as behaviors for increasing and decreasing speed. Engine is a separate object consisting of Capacity and Fuel Type.

Create at least 2 related classes; each class has its own attributes and methods, as well as getter-setter methods.

- Draw a UML diagram of these classes (include multiplicity).
- Apply at least 1 defensive programming pattern you have learned.
- Consider: does your design also need a Uses-A relation (e.g., a method that receives another object as a parameter without storing it)?

Questions?

Thank You

